

Proyecto 1 (versión XMLHttpRequest) – Consumo de JSON con AJAX y Bootstrap

Objetivo del proyecto para el alumno

- Hacer **peticiones AJAX** con `XMLHttpRequest` al servidor.
- Procesar la respuesta en **JSON** y mostrarla dinámicamente.
- Actualizar una **tabla de Bootstrap** sin recargar la página.
- Entender la **comunicación cliente → servidor → cliente** paso a paso.

Lo que se espera que haga el alumno:

1. Iniciar un servidor Node.js con Express que devuelva datos de productos en JSON.
2. Crear una página web con **Bootstrap** para mostrar los productos en una **tabla**.
3. Usar **XMLHttpRequest** para solicitar los datos al servidor.
4. Añadir botones para **cargar todos** y **filtrar por categoría**.

1. Servidor con Express (Node.js)

Este código es igual que el proyecto original.

```
// server.js
const express = require('express');
const cors = require('cors');
const app = express();

app.use(cors());

// Datos simulados
const productos = [
  { id: 1, nombre: "Laptop", categoria: "Electrónica", precio: 1200 },
  { id: 2, nombre: "Mouse", categoria: "Electrónica", precio: 25 },
  { id: 3, nombre: "Teclado", categoria: "Electrónica", precio: 45 },
  { id: 4, nombre: "Leche", categoria: "Alimentos", precio: 2 },
  { id: 5, nombre: "Pan", categoria: "Alimentos", precio: 1.5 }
];

// Ruta principal: devuelve todos los productos
app.get('/api/productos', (req, res) => {
  res.json(productos);
});

// Ruta con filtro por categoría
```

```

app.get('/api/productos/:categoria', (req, res) => {
  const categoria = req.params.categoria.toLowerCase();
  const filtrados = productos.filter(p => p.categoria.toLowerCase()
=== categoria);
  res.json(filtrados);
});

// Iniciar servidor
const PORT = 3000;
app.listen(PORT, () => console.log(`Servidor escuchando en
http://localhost:${PORT}`));

```

Ejecutar servidor:

```
node server.js
```

2. Cliente HTML + Bootstrap

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Productos AJAX - XMLHttpRequest</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.
min.css" rel="stylesheet">
</head>
<body class="container py-4">
  <h1>Lista de Productos</h1>

  <!-- Filtros -->
  <div class="mb-3">
    <input type="text" id="categoria" class="form-control"
placeholder="Filtrar por categoría">
    <button id="btnFiltrar" class="btn btn-primary mt-
2">Filtrar</button>
    <button id="btnCargarTodos" class="btn btn-secondary mt-
2">Cargar Todos</button>
  </div>

  <!-- Tabla de productos -->
  <table class="table table-striped">
    <thead>
      <tr>
        <th>ID</th>
        <th>Nombre</th>
        <th>Categoría</th>
        <th>Precio</th>
      </tr>
    </thead>
    <tbody id="tablaProductos"></tbody>
  </table>

  <script src="script.js"></script>
</body>
</html>

```

3. JavaScript usando XMLHttpRequest

```
const tabla = document.getElementById('tablaProductos');
const btnCargarTodos = document.getElementById('btnCargarTodos');
const btnFiltrar = document.getElementById('btnFiltrar');
const inputCategoria = document.getElementById('categoria');

const API_BASE = 'http://localhost:3000/api/productos';

// Función para mostrar productos en la tabla
function mostrarProductos(productos) {
    tabla.innerHTML = '';
    productos.forEach(p => {
        const row = document.createElement('tr');
        row.innerHTML = `
            <td>${p.id}</td>
            <td>${p.nombre}</td>
            <td>${p.categoria}</td>
            <td>${p.precio}</td>
        `;
        tabla.appendChild(row);
    });
}

// Cargar todos los productos
btnCargarTodos.addEventListener('click', () => {
    const xhr = new XMLHttpRequest();
    xhr.open('GET', API_BASE, true);

    xhr.onload = function() {
        if(xhr.status === 200){
            const data = JSON.parse(xhr.responseText);
            mostrarProductos(data);
        } else {
            alert('Error al cargar productos');
        }
    };

    xhr.onerror = function() {
        alert('Error en la petición AJAX');
    };

    xhr.send();
});

// Filtrar por categoría
btnFiltrar.addEventListener('click', () => {
    const categoria = inputCategoria.value.trim();
    if(!categoria){
        alert('Introduce una categoría');
        return;
    }

    const xhr = new XMLHttpRequest();
    xhr.open('GET', `${API_BASE}/${encodeURIComponent(categoria)}`,
true);

    xhr.onload = function() {
        if(xhr.status === 200){
            const data = JSON.parse(xhr.responseText);
```

```

        mostrarProductos(data);
    } else {
        alert('Error al filtrar productos');
    }
};

xhr.onerror = function() {
    alert('Error en la petición AJAX');
};

xhr.send();
});

```

4. Funcionamiento del proyecto

1. Ejecutar el servidor: `node server.js`
2. Abrir `index.html` en el navegador.
3. Hacer click en **Cargar Todos** → se muestra la tabla con todos los productos.
4. Escribir una categoría y hacer click en **Filtrar** → se muestran solo los productos de esa categoría.
5. Todo sucede **sin recargar la página**, usando **XMLHttpRequest** clásico.

5. Ejercicios propuestos para los alumnos

1. Cambiar el código para mostrar **mensaje "Cargando..."** mientras llega la respuesta del servidor.
2. Agregar un **campo de precio máximo** para filtrar productos.
3. Crear un **botón para ordenar por precio ascendente** usando los datos recibidos en `XMLHttpRequest`.
4. Agregar manejo de errores si la categoría no existe y mostrar un mensaje en la página.
5. Modificar el servidor para **añadir productos nuevos** y permitir cargar los productos actualizados con AJAX.